FRONTEND THEORY ASSIGNMENT - 3

PRIYANSH AGARWAL
NC-39
22515000104

1. What is the difference between frontend and backend JavaScript?

JavaScript is used on both sides of a web application, but the two roles are very different.

Frontend JavaScript runs in the user's browser. It builds the interactive part of a website: handling clicks, validating forms, showing menus and sliders, updating content on the screen, and calling APIs to fetch data. It has access to browser objects such as window, document and localStorage.

Backend JavaScript runs on a server, most commonly with Node.js. It receives requests from the browser, applies business logic, talks to databases, handles login and authentication, reads and writes files, and sends responses back. It has no DOM, but it can use modules like fs and http, and frameworks like Express.

In short: frontend JavaScript controls what the user sees and does, while backend JavaScript controls how data is processed and stored. For example, when a user fills a signup form, frontend JavaScript checks that the email is valid and the backend JavaScript saves the account in the database.

2. What is DOM?

DOM stands for Document Object Model. It is a programming interface created by the browser when a page loads. The DOM represents the HTML document as a tree of objects (called nodes), where the document is the root, followed by html, head, body and all the elements inside them.

JavaScript uses the DOM to select elements, change their text, attributes and styles, add or remove elements, and react to user actions, all without reloading the page. The DOM is therefore the link between HTML and JavaScript.

```
const heading = document.getElementById("title");
heading.textContent = "Hello, DOM!";
heading.style.color = "blue";

const p = document.createElement("p");
p.textContent = "New paragraph";
document.body.appendChild(p);
```

3. What is an event in JavaScript?

An event is something that happens in the browser which JavaScript can detect and respond to. Events can be triggered by the user (click, key press, mouse move, form submit, scroll) or by the browser itself (page load, window resize, image loaded).

To respond to an event, we attach an event listener to an element using addEventListener(). The listener function receives an event object that carries details such as the target element,

the key pressed, or the mouse coordinates.

```
const btn = document.querySelector("#btn");
btn.addEventListener("click", function (e) {
  console.log("Button clicked:", e.target);
});
```

Common event categories are mouse events (click, dblclick, mouseover), keyboard events (keydown, keyup), form events (submit, change, input), window events (load, resize) and touch events. Events make web pages interactive.

4. What is the event loop in JavaScript?

JavaScript is single-threaded, so it can execute only one piece of code at a time. Even so, it can handle timers, network requests and user events without freezing the page. This is made possible by the event loop.

The main parts involved are:

Call Stack: where functions are executed one by one.
Web APIs / Node APIs: where asynchronous tasks such as setTimeout, fetch and DOM events are handled outside the main thread.
Callback (Task) Queue: where callbacks wait once their task finishes.
Microtask Queue: where promise callbacks (.then, async/await) wait. It has higher priority than the callback queue.

The event loop keeps checking whether the call stack is empty. When it is, the loop takes the next callback from the microtask queue (first) or the task queue and pushes it onto the stack to run.

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
// Output: 1, 4, 3, 2
```

5. What is hoisting in JavaScript?

Hoisting is JavaScript's behaviour of processing declarations before the code runs, which makes them appear as if they were moved to the top of their scope. Only the declaration is hoisted, not the assigned value.

var: hoisted and initialised with undefined, so using it early does not throw an error.
let and const: hoisted but not initialised. They stay in the Temporal Dead Zone (TDZ) until the line where they are declared, and accessing them earlier throws a ReferenceError.
Function declarations: fully hoisted, so they can be called before they are written.
Function expressions and arrow functions: follow the rules of the variable that stores them, so they cannot be called early.

```
console.log(a);   // undefined
var a = 10;
```

```
sayHi();          // works
function sayHi() { console.log("Hi"); }

console.log(b);   // ReferenceError (TDZ)
let b = 5;
```

6. What is event bubbling?

Event bubbling is the way events travel through the DOM. When an event occurs on an element, it first runs the handlers on that element (the target) and then bubbles up to its parent, grandparent, and so on until it reaches document.

For example, if a button is inside a div and both have click listeners, clicking the button runs the button's handler first and then the div's handler.

```
<div id="parent">
  <button id="child">Click</button>
</div>

parent.addEventListener("click", () => console.log("Parent"));
child.addEventListener("click", () => console.log("Child"));
// Click on button prints: Child, then Parent
```

Bubbling can be stopped with event.stopPropagation(). It also enables event delegation, where a single listener on a parent handles events for many child elements, saving memory and working for elements added later. The opposite phase, where the event travels down from the root to the target, is called event capturing.

7. What is the difference between named function, anonymous function and arrow function?

Named function: a function that has its own name. It can be called by that name, it is hoisted when written as a declaration, and the name appears in stack traces, which helps debugging.

```
function add(a, b) {
  return a + b;
}
```

Anonymous function: a function without a name. It is usually stored in a variable or passed directly as an argument, such as a callback. It is not hoisted when stored in a variable.

```
const add = function (a, b) {
  return a + b;
};
setTimeout(function () { console.log("Done"); }, 1000);
```

Arrow function: a shorter syntax introduced in ES6 using =>. It has no this of its own (it uses the this of the surrounding scope), has no arguments object, and cannot be used as a constructor. If the body is a single expression, the return is implicit.

```
const add = (a, b) => a + b;
const square = n => n * n;
```

8. Explain for-of and for-in loop

for-of loops over the values of an iterable such as an array, string, Map or Set. It is the simplest way to read each element without using indexes. It does not work on plain objects.

for-in loops over the enumerable keys (property names) of an object. On an array it gives the index positions as strings, and it may also include inherited properties, so it is best suited for objects rather than arrays.

```
const names = ["Amit", "Riya", "Karan"];

for (const n of names) console.log(n);   // Amit, Riya, Karan
for (const i in names) console.log(i);   // 0, 1, 2

const user = { name: "Riya", age: 20 };
for (const key in user) {
  console.log(key, user[key]);          // name Riya, age 20
}
```

Summary: use for-of when you need values and for-in when you need property names of an object.

9. Explain any five array methods

1. push()

Adds one or more elements to the end of an array and returns the new length. It modifies the original array.

```
const fruits = ["apple", "banana"];
fruits.push("mango");   // ["apple", "banana", "mango"]
```

2. pop()

Removes the last element and returns it. It modifies the original array.

```
fruits.pop();          // returns "mango"
```

3. shift()

Removes the first element and returns it. The remaining elements move one position to the left.

```
fruits.shift();        // returns "apple"
```

4. unshift()

Adds one or more elements to the beginning of an array and returns the new length.

```
fruits.unshift("grapes");   // ["grapes", "banana"]
```

5. map()

Creates a new array by calling a function on every element of the original array. The original array is not changed.

```
const nums = [1, 2, 3];
const doubled = nums.map(n => n * 2);   // [2, 4, 6]
```

Other useful methods include filter(), reduce(), forEach(), find(), slice() and splice().

10. What is a callback method?

A callback function is a function that is passed as an argument to another function and is called later, usually after some task has finished or an event has occurred. The function that receives it decides when to execute it.

Callbacks are very common in JavaScript because many operations take time, such as timers, reading files, fetching data from a server and handling user events. Instead of waiting and blocking the program, JavaScript continues with other code and runs the callback when the result is ready.

```
function greet(name, callback) {
  console.log("Hello " + name);
  callback();
}
greet("Priyansh", function () {
  console.log("Welcome to the course!");
});

setTimeout(() => console.log("Runs after 2 seconds"), 2000);
[1, 2, 3].forEach(n => console.log(n));   // callback used by forEach
```

Callbacks can become hard to read when many are nested inside each other (called callback hell). Modern JavaScript solves this with Promises and async/await.